

Linux Server Deployment Automation

Zero-touch provisioning of a 70+ node Rocky Linux workstation fleet — from bare metal to fully configured host — using PXE network boot, Bash bootstrap scripts, idempotent Ansible playbooks, and AWX with Git-backed source control.

PXE / TFTP / DHCP	Kickstart	Rocky Linux 9	Ansible
Ansible AWX	Bash	systemd	Git (SCM)

Environment	Fleet scale	Reference set
Northstar Animation Studios — Linux pipeline	70+ identical artist / render nodes	15 nodes documented end-to-end

Summary. Provisioning 70+ identical Linux workstations by hand is slow, inconsistent, and impossible to audit. This project replaces manual installs with a two-stage automated pipeline. **Stage one** uses a single dedicated PXE server (DHCP + TFTP + HTTP) to network-boot each machine and run an unattended Rocky Linux install driven by a Kickstart file. **Stage two** hands the freshly installed host to Ansible — orchestrated through AWX and a Git-backed playbook repository — which applies users, firewall policy, services, applications, environment, and monitoring agents idempotently. The result is a reproducible, version-controlled path from a powered-off bare-metal box to a production-ready workstation, with no operator touching the keyboard of the target node.

1 — Architecture Overview

The environment is split into two logical planes. The **provisioning plane** is a single dedicated server responsible for getting an operating system onto bare metal. The **configuration plane** is the AWX control node, which pulls playbooks from Git and converges every host to a known desired state. Both planes sit on the studio's flat production network behind the gateway firewall.

End-to-end flow

1	Power on / PXE	Node NIC sends a DHCP discover; firmware is set to boot from network (UEFI PXE).
2	Lease + boot file	PXE server's DHCP offers an IP and points the client at the bootloader over TFTP.
3	Bootloader + kernel	shim / GRUB, kernel (vmlinuz) and initrd transfer over TFTP; installer starts.
4	Kickstart install	Installer fetches the Kickstart over HTTP; Rocky Linux installs unattended.
5	First boot + bootstrap	Node reboots into the OS; a Bash bootstrap registers it and prepares Ansible access.
6	Ansible / AWX converge	AWX runs the role-based playbooks from Git against the node until desired state is met.
7	Production-ready	Users, firewall, apps, env, and monitoring agents are live; node joins the fleet.

Key hosts

Role	Hostname	Address	Purpose
PXE / install server	ns-pxe01	10.20.0.10	DHCP + TFTP + HTTP (Kickstart); OS provisioning
AWX control node	ns-awx01	10.20.0.11	Web UI, job templates, Git project sync, inventory
Gateway firewall	ns-fw01	10.20.0.1	Network gateway / egress policy
Workstation fleet	ns-wks-01...70	10.20.20.0/24	Identical artist / render nodes (targets)

2 — Stage One: PXE Provisioning Plane

A single dedicated Linux box (ns-pxe01) runs the complete provisioning stack. Keeping DHCP, TFTP, and HTTP on one server makes the boot chain self-contained and easy to reason about: one machine answers the network boot request, serves the bootloader and kernel, and hosts both the OS payload and the unattended install recipe.

2.1 — Components

Service	Software	Role in the boot chain
DHCP	dnsmasq / ISC dhcpd	Assigns the client an IP and hands over next-server + boot filename.
TFTP	dnsmasq / tftp-server	Serves the bootloader (shim/GRUB), kernel and initrd.
HTTP	Apache (httpd)	Serves the extracted Rocky ISO tree and the Kickstart file.
Bootloader	shim + GRUB2 (UEFI)	Loads the installer kernel; passes the Kickstart URL as a kernel arg.

2.2 — Preparing the server

Install the services, open the required ports, and stage the OS content. The Rocky ISO is loop-mounted and copied into the Apache document root so the installer can pull packages over HTTP.

```
# packages
dnf install -y dnsmasq httpd syslinux

# firewall: DHCP, TFTP, HTTP
firewall-cmd --add-service={dhcp,tftp,http} --permanent
firewall-cmd --reload

# stage the Rocky 9 install tree under the web root
mount -o loop Rocky-9-x86_64-dvd.iso /mnt/iso
mkdir -p /var/www/html/rocky9
cp -a /mnt/iso/. /var/www/html/rocky9/

# UEFI boot files into the TFTP root
mkdir -p /var/lib/tftpboot/uefi
cp /mnt/iso/EFI/B00T/{B00TX64.EFI,grubx64.efi} /var/lib/tftpboot/uefi/
cp /mnt/iso/images/pxeboot/{vmlinuz,initrd.img} /var/lib/tftpboot/uefi/
```

2.3 — DHCP + TFTP (dnsmasq)

A compact dnsmasq config covers both DHCP and TFTP. It points UEFI clients at the GRUB bootloader and enables the built-in TFTP server.

```
# /etc/dnsmasq.conf
interface=enol
bind-interfaces

# DHCP lease range for the workstation VLAN
dhcp-range=10.20.20.50,10.20.20.250,12h
dhcp-option=3,10.20.0.1 # gateway
dhcp-option=6,10.20.0.1 # DNS

# point UEFI PXE clients at the GRUB bootloader
dhcp-boot=uefi/B00TX64.EFI

# built-in TFTP
enable-tftp
tftp-root=/var/lib/tftpboot
```

2.4 — Kickstart: the unattended install

The Kickstart file is the heart of stage one. It answers every installer prompt — disk layout, network, timezone, root credentials, the package set, and a tiny %post hook that lays the groundwork for stage two by enabling SSH and creating the automation user Ansible will connect as.

```
# /var/www/html/ks/workstation.ks
text
lang en_US.UTF-8
keyboard us
timezone Asia/Kolkata --utc

# pull packages over HTTP from this server
url --url="http://10.20.0.10/rocky9"

network --bootproto=dhcp --device=link --activate
rootpw --iscrypted $6$REDACTED

# wipe and auto-partition
clearpart --all --initlabel
autopart --type=lvm
bootloader --location=mbr
reboot

%packages
@^minimal-environment
openssh-server
python3
%end

%post --log=/root/ks-post.log
# create the automation account for Ansible
useradd -m -G wheel ansible
echo 'ansible ALL=(ALL) NOPASSWD:ALL' > /etc/sudoers.d/ansible
mkdir -p /home/ansible/.ssh && chmod 700 /home/ansible/.ssh
curl -s http://10.20.0.10/keys/ansible.pub >> /home/ansible/.ssh/authorized_keys
chmod 600 /home/ansible/.ssh/authorized_keys
chown -R ansible:ansible /home/ansible/.ssh
%end
```

Because the install is fully described in a file, every node comes up identically — same partition scheme, same base packages, same automation account and SSH key. There is no per-machine clicking and no drift introduced at install time.

3 — Stage Two: Configuration Plane

Once a node reboots into a clean Rocky install with the `ansible` user and key already in place, configuration management takes over. The desired state of every workstation lives in a Git-backed Ansible repository, applied through AWX.

3.1 — Bash bootstrap

A small bootstrap script handles the handful of things that are simplest to do imperatively before Ansible runs: confirming connectivity, pinning the hostname from a MAC/serial lookup, and registering the node into the inventory. It is intentionally thin — anything that defines long-term state belongs in a playbook, not the script.

```
#!/usr/bin/env bash
# bootstrap.sh – run once on first boot
set -euo pipefail

CONTROL="10.20.0.11" # AWX / control node
ROLE="workstation"

# derive a stable hostname from the primary MAC
MAC=$(cat /sys/class/net/enol/address | tr -d ':')
hostnamectl set-hostname "ns-wks-${MAC: -4}"

# make sure python + ssh are present for Ansible
dnf install -y python3 openssh-server
systemctl enable --now sshd

# announce readiness to the control node
curl -s -X POST "http://${CONTROL}:8080/register" \
  -d "host=$(hostname)&role=${ROLE}"
echo "bootstrap complete: $(hostname)"
```

3.2 — Inventory (/etc/ansible/hosts)

Hosts are grouped statically in `/etc/ansible/hosts`. The fleet runs to 70+ nodes; the reference inventory below shows the first 15, which is enough to exercise every group and variable pattern the full fleet uses.

```
# /etc/ansible/hosts
[workstations]
ns-wks-01 ansible_host=10.20.20.1
ns-wks-02 ansible_host=10.20.20.2
ns-wks-03 ansible_host=10.20.20.3
ns-wks-04 ansible_host=10.20.20.4
ns-wks-05 ansible_host=10.20.20.5
ns-wks-06 ansible_host=10.20.20.6
ns-wks-07 ansible_host=10.20.20.7
ns-wks-08 ansible_host=10.20.20.8
ns-wks-09 ansible_host=10.20.20.9
ns-wks-10 ansible_host=10.20.20.10
ns-wks-11 ansible_host=10.20.20.11
ns-wks-12 ansible_host=10.20.20.12
ns-wks-13 ansible_host=10.20.20.13
ns-wks-14 ansible_host=10.20.20.14
ns-wks-15 ansible_host=10.20.20.15
# ... ns-wks-16 through ns-wks-70

[render:children]
workstations

[all:vars]
ansible_user=ansible
ansible_python_interpreter=/usr/bin/python3
```

3.3 — Role-based playbook

The site playbook applies a set of focused roles. Each role owns one concern and is written to be idempotent — re-running it on an already-configured node makes no changes and reports ok rather than changed.

```
# site.yml
- hosts: workstations
  become: true
  roles:
    - common # base packages, NTP, repos, timezone
    - users # studio accounts, groups, SSH keys
    - firewall # firewalld zones + service rules
    - apps # DCC tools, render client, env vars
    - monitoring # node_exporter + log shipping agent
```

3.4 — Idempotent role example

The roles lean on Ansible's declarative modules (user, firewalld, dnf, systemd) rather than shell commands, so state is checked before it is changed. The example below configures the firewall and the monitoring agent — both safe to run on every converge.

```
# roles/firewall/tasks/main.yml
- name: Allow SSH and monitoring scrape
  ansible.posix.firewalld:
    service: "{{ item }}"
    permanent: true
    immediate: true
    state: enabled
  loop:
    - ssh
    - node-exporter

# roles/monitoring/tasks/main.yml
- name: Install node_exporter
  ansible.builtin.dnf:
    name: golang-github-prometheus-node-exporter
    state: present

- name: Ensure node_exporter is enabled and running
  ansible.builtin.systemd:
    name: node_exporter
    enabled: true
    state: started
```

What 'idempotent' buys

- **Safe re-runs.** The same playbook can run on a fresh node or a 200-day-old node with identical, predictable results.
- **Drift correction.** If someone changes a firewall rule by hand, the next converge quietly puts it back.
- **Honest reporting.** The changed count is a real signal — a non-zero value on a stable node means something actually moved.

4 — AWX & Git-Backed Source Control

AWX (the upstream of Ansible Automation Platform) puts a web UI, RBAC, scheduling, and audit logging in front of the playbooks. Critically, AWX does not store playbooks itself — it syncs them from a Git repository, so the repo remains the single source of truth and every run is traceable to a specific commit.

4.1 — How the pieces connect

AWX object	Bound to	What it does
Project	GitHub repo (SCM)	Clones / pulls playbooks on demand or on a schedule.
Inventory	/etc/ansible/hosts	Imported via an inventory source; groups the 70+ nodes.
Credential	SSH key (ansible user)	Injected at runtime; never written into the repo.
Job Template	site.yml + inventory	The runnable unit — launch, schedule, or trigger it.
Schedule	Job Template	Nightly converge keeps the fleet in desired state.

4.2 — GitOps workflow

A change to the fleet is a change to the repository. The loop is small and auditable:

Edit	A role or variable is changed on a branch and opened as a pull request.
Review + merge	The PR is reviewed and merged to main.
Project sync	AWX pulls the new commit (manually or on the next scheduled sync).
Launch	The Job Template runs site.yml against the target inventory.
Audit	AWX records who launched what, against which commit, with full job output.

4.3 — Rolling out to the fleet

For changes that carry risk, the Job Template runs in batches using a serial strategy so a bad change never hits all 70 nodes at once. A single converge across the whole fleet — or a targeted subset via `--limit` — is one click from the AWX UI or one API call.

```
# site.yml — limit blast radius during rollout
- hosts: workstations
  serial: "20%" # 20% of the fleet at a time
  max_fail_percentage: 5 # abort the rollout if >5% fail
  become: true
  roles:
    - common
    - users
    - firewall
    - apps
    - monitoring
```


5 — Outcomes

Before	After
Manual OS install per machine, ~45 min of operator time each.	Unattended PXE + Kickstart install; operator time per node ≈ 0 .
Configuration done by hand; silent drift between nodes.	Declarative Ansible roles; drift corrected on every converge.
No record of what changed, when, or by whom.	Every change is a Git commit; every run is logged in AWX.
Scaling the fleet meant proportionally more manual labour.	Adding nodes is inventory edits; effort is flat with scale.
Onboarding a new build required tribal knowledge.	The repo is the documentation — reproducible from bare metal.

Technical stack

Layer	Technology
OS	Rocky Linux 9
Network boot	PXE (UEFI) — dnsmasq (DHCP + TFTP), Apache (HTTP)
Unattended install	Kickstart
Bootstrap	Bash
Configuration management	Ansible (role-based, idempotent)
Inventory	/etc/ansible/hosts (static groups)
Orchestration / UI	Ansible AWX
Source control	Git (GitHub), SCM-synced AWX Project
Service management	systemd
Monitoring agent	Prometheus node_exporter

Names, hostnames, and addresses in this document are anonymized for portfolio use. The architecture, tooling, and workflow reflect a real production deployment.